

UNIVERSITATEA DIN BUCUREȘTI  
FACULTATEA DE  
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

# SISTEM DE DETECTARE A INTRUZIUNILOR UTILIZÂND ÎNVĂȚAREA PRIN RECOMPENSĂ

Absolvent

Igescu Rareș-Andrei

Coordonator științific

Conf. univ. dr. Ciprian Păduraru

București, iunie 2026

# Abstract

Lorem ipsum

# Rezumat

Lorem ipsum

# Cuprins

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducere</b>                                              | <b>3</b>  |
| 1.1      | Context și motivație . . . . .                                  | 3         |
| 1.2      | Contribuția lucrării . . . . .                                  | 3         |
| <b>2</b> | <b>Fundamente teoretice și cercetări conexe</b>                 | <b>4</b>  |
| 2.1      | Sisteme de detecție bazate pe semnături (SIDS) . . . . .        | 4         |
| 2.2      | Sisteme de detecție bazate pe anomalii (AIDS) . . . . .         | 5         |
| 2.3      | Fundamentele Învățării prin Recompensă (RL) . . . . .           | 6         |
| 2.3.1    | Procese Decizionale Markov (MDP) . . . . .                      | 6         |
| 2.3.2    | Ecuția Bellman și Deep Q-Networks (DQN) . . . . .               | 7         |
| 2.4      | Stadiul actual al cercetării . . . . .                          | 8         |
| <b>3</b> | <b>Dezvoltarea Sistemului de Detecție</b>                       | <b>9</b>  |
| 3.1      | Pregătirea și preprocesarea datelor experimentale . . . . .     | 9         |
| 3.2      | Proiectarea mediului de antrenament . . . . .                   | 10        |
| 3.2.1    | Spațiul stărilor și spațiul acțiunilor . . . . .                | 10        |
| 3.2.2    | Proiectarea funcției de recompensă . . . . .                    | 11        |
| 3.2.3    | Episoadele de antrenament și generalizarea . . . . .            | 11        |
| 3.3      | Algoritmul Deep Q-Network (DQN) . . . . .                       | 12        |
| 3.3.1    | De la ecuația Bellman la funcția de pierdere . . . . .          | 12        |
| 3.3.2    | Memoria de reluare ( <i>Experience Replay</i> ) . . . . .       | 13        |
| 3.3.3    | Rețeaua țintă ( <i>Target Network</i> ) . . . . .               | 14        |
| 3.3.4    | Politica de explorare $\varepsilon$ -greedy . . . . .           | 14        |
| 3.4      | Arhitectura și Configurarea Agentului . . . . .                 | 15        |
| 3.4.1    | Arhitectura rețelei neuronale . . . . .                         | 15        |
| 3.4.2    | Sinteza hiperparametrilor . . . . .                             | 16        |
| 3.5      | Rezultatele Antrenamentului și Evaluarea Performanței . . . . . | 17        |
| <b>4</b> | <b>Concluzii</b>                                                | <b>19</b> |
| <b>5</b> | <b>Bibliografie</b>                                             | <b>20</b> |

# 1. Introducere

## 1.1 Context și motivație

Rețelele informatice au evoluat în ultimele decenii, generând volume masive de trafic și complexitate sporită în identificarea amenințărilor cibernetice. Inteligența artificială și metodele de învățare prin Reinforcement Learning (RL) au demonstrat un potențial semnificativ în dezvoltarea sistemelor automate de detecție a intruziunilor (IDS). Majoritatea abordărilor tradiționale se concentrează pe aplicarea unor seturi de reguli statice, fără a ține cont de necesitatea adaptării dinamice la atacuri noi într-un mediu de rețea real.

Pentru a rezolva această problemă, paradigma rețelelor neuronale profunde de tip Deep Q-Network (DQN) devine tot mai relevantă. Un agent DQN poate evalua continuu starea rețelei, învățând autonom politici decizionale optime pentru a distinge comportamentele malițioase de cele legitime în timp real.

## 1.2 Contribuția lucrării

Prezenta lucrare de licență își propune dezvoltarea, implementarea și evaluarea unui Sistem de Detecție a Intruziunilor (IDS) adaptiv, bazat pe o arhitectură Deep Q-Network (DQN).

Pentru a atinge acest scop principal, au fost stabilite următoarele obiective specifice:

- **1. Modelarea mediului de rețea ca un Proces Decizional Markov (MDP):** Proiectarea unei funcții de recompensă personalizate care să permită agentului să găsească un echilibru optim între securitate și disponibilitatea rețelei.
- **2. Implementarea agentului RL:** Dezvoltarea arhitecturii DQN utilizând rețele neuronale profunde pentru aproximarea funcției de valoare, capabilă să proceseze spațiul continuu al caracteristicilor de trafic.
- **3. Antrenarea și validarea pe date moderne:** Utilizarea setului de date **CICIDS2017** (Canadian Institute for Cybersecurity), care conține trafic benign actualizat și cele mai comune familii de atacuri moderne, asigurând astfel relevanța practică a sistemului.

## 2. Fundamente teoretice și cercetări conexe

### 2.1 Sisteme de detecție bazate pe semnături (SIDS)

Sistemele de detecție bazate pe semnături reprezintă o metodă tradițională în securitatea rețelor, funcționând pe un principiu similar programelor antivirus clasice. Aceste sisteme monitorizează traficul de rețea și compară conținutul pachetelor cu o bază de date predefinită de ”semnături” (reguli) asociate atacurilor cunoscute.

Din punct de vedere algoritmic, eficiența acestui proces depinde de viteza cu care se efectuează potrivirea modelelor (*pattern matching*). Pentru a face față volumului mare de date în timp real, majoritatea soluțiilor consacrate utilizează algoritmul **Aho-Corasick** [1]. Acesta construiește un automat finit determinist din baza de date de semnături, permițând căutarea simultană a tuturor modelelor într-o singură parcurgere a pachetului de date, garantând astfel o complexitate liniară  $\mathcal{O}(n)$  care este totodată independentă de numărul de semnături ( $k$ ).

Semnăturile menționate anterior reprezintă, practic, amprenta digitală a unui atac, definită printr-un șir specific de bytes. Cel mai reprezentativ exemplu industrial este **Snort**, care utilizează reguli de forma:

*Dacă pachetul vine de la IP-ul  $X$  și conține șirul hexazecimal  $Y$ , atunci generează o alertă.*

Deși sistemele bazate pe semnături oferă o rată scăzută de alarme false (*False Positives*) pentru alarmele cunoscute, rigiditatea acestora constituie o mare breșă în contextul contemporan al securității:

- **Incapacitatea de a detecta atacuri Zero-Day:** Deoarece funcționează exclusiv pe baza unei liste negre (*blacklist*), orice atac nou apărut, pentru care nu a fost încă scrisă o semnătură, va trece neobservat.
- **Vulnerabilitate la Polimorfism:** Atacatorii pot modifica ușor semnătura unui malware (prin criptare sau schimbarea unor biți ne semnificativi) fără a-i altera funcționalitatea malițioasă. Această tehnică face ca semnătura din baza de date să nu se mai potrivească exact, păcălind algoritmul de pattern matching.
- **Mentenanță costisitoare:** Baza de date necesită actualizări manuale și continue, proces adesea prea lent față de viteza de propagare a noilor vectori de atac.

Aceste limitări necesită tranziția către paradigme de detecție euristice și comportamentale, precum

**Sistemele Bazate pe Anomalii (AIDS)**, capabile să generalizeze și să identifice intenții malițioase necunoscute anterior.

## 2.2 Sisteme de detecție bazate pe anomalii (AIDS)

Acest model, propus inițial de **Denning** [2], reprezintă un sistem expert universal pentru detecția intruziunilor în timp real, bazat pe ipoteza că abuzurile informatice pot fi identificate prin monitorizarea anomaliilor din jurnalele de activitate. Prin utilizarea profilurilor statistice și a metricilor de comportament, sistemul învață interacțiunile normale dintre utilizatori și resurse pentru a semnaliza devierile suspecte, oferind un cadru adaptabil oricărei platforme, indiferent de tipul vulnerabilităților sau al atacurilor.

Pentru a putea detecta cu succes traficul suspect sau malițios, sistemul trebuie mai întâi să învețe acțiuni normale. Astfel, procesul de funcționare al unui sistem bazat pe anomalii poate fi împărțit în două:

- **Faza de antrenare:** Sistemul analizează un volum mare de trafic (protocoale utilizate des, lățime de bandă) care poate fi considerat normal pentru a stabili un profil de referință (cunoscut drept *baseline*) al activității.
- **Faza de detecție:** Orice deviație de la acest standard va fi marcată de sistem ca anomalie și va ridica o alertă la nivelul traficului.

Avantajul major al acestor sisteme este capacitatea de a detecta atacuri de tip **Zero-Day** deoarece sistemul nu are nevoie să cunoască de dinainte amprenta atacului; este suficient ca traficul malițios să se comporte diferit față de cel normal.

Totuși, implementarea clasică a sistemelor AIDS vine cu o provocare, anume rata ridicată a alarmelor false (*False-Positive Rate*). Rețelele moderne sunt dinamice, cu trafic variabil, iar un comportament atipic, dar legitim (trimiterea bruscă a mai multor fișiere de dimensiuni mari) poate fi marcat ca trafic malițios.

Pentru a rezolva această problemă a alarmelor false și a crea un sistem capabil să se adapteze continuu la dinamica rețelei, soluțiile moderne recurg la algoritmi avansați de inteligență artificială (AI). Dintre aceștia, o paradigmă care permite sistemului să învețe direct din consecințele deciziilor sale este **Învățarea prin Recompensă (Reinforcement Learning)**.

## 2.3 Fundamentele Învățăării prin Recompensă (RL)

Conform fundamentelor stabilite de **Sutton și Barto** [3], învățarea prin recompensă constă în capacitatea unui agent de a descoperi ce acțiuni trebuie întreprinse în anumite situații, cu scopul fundamental de a maximiza un semnal numeric de recompensă.

Multe sisteme reale nu au soluții perfecte, ci doar acțiuni mai bune în funcție de experiența acumulată până în acel moment. Un agent bazat pe învățarea prin recompensă încearcă să învețe cum să acționeze optim într-un mediu necunoscut. În contextul securității cibernetice, agentul joacă rolul sistemului de detecție, iar mediul din care se poate antrena este reprezentat de traficul rețelei monitorizate. Ciclul generic pentru un agent RL poate fi analizat în figura 2.1.

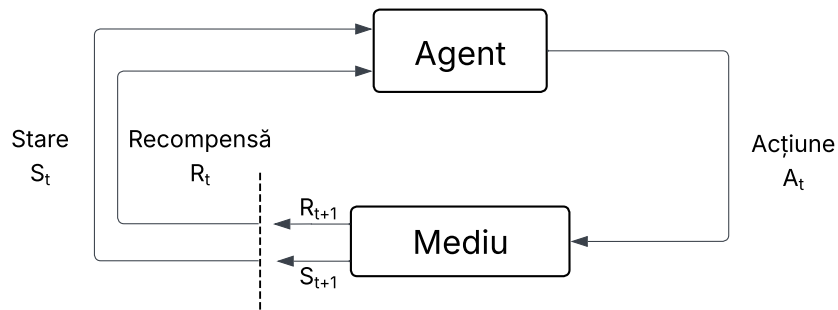


Figura 2.1: Ciclul de interacțiune Agent-Mediu în sistemul propus

### 2.3.1 Procese Decizionale Markov (MDP)

Procesele Markov sunt procese stochastice în care viitorul depinde doar de prezent. Mai precis, un proces Markov îndeplinește proprietatea Markov doar dacă probabilitatea de a ajunge într-o stare viitoare depinde exclusiv de starea curentă. Dacă spațiile de stări sunt finite, atunci acesta se numește proces de decizie Markov finit.

$$Pr(s_t | s_{t-1}, \dots, s_0) = Pr(s_t | s_{t-1})$$

Ecuția 2.1 Procese Markov de ordinul I

Această relație definește „independența de istoric”, sugerând că starea  $s_{t-1}$  încapsulează toate informațiile relevante din trecut necesare pentru a prezice starea viitoare  $s_t$ . În mod implicit, procesele

Markov se referă la procesele de ordinul I. Totuși, există situații complexe în care dependența se extinde asupra mai multor stări anterioare, ducând la procese de ordin superior:

$$Pr(s_t | s_{t-1}, \dots, s_0) = Pr(s_t | s_{t-1}, s_{t-2})$$

### Ecuția 2.2 Procese Markov de ordinul II

Din puncte de vedere formal, un proces de decizie Markov este definit de următorul tuplu  $(S, A, P, R, \gamma)$ , unde:

- $S$  reprezintă mulțimea finită de stări;
- $A$  reprezintă mulțimea finită de acțiuni;
- $P(s' | s, a)$  este probabilitatea de a ajunge în starea  $s'$  după ce agentul alege acțiunea  $a$  în starea  $s$ ;
- $R(s, a, s')$  este recompensa primită pentru tranziția  $(s \rightarrow s')$ ;
- $\gamma \in [0, 1]$  este factorul de discount, care controlează importanța viitorului.

În contextul detecției anomaliilor în trafic, proprietatea Markov presupune că pachetul curent (sau fereastra de trafic actuală) conține suficiente informații pentru a decide dacă este un atac, fără a fi necesară analiza întregului istoric al conexiunii de la inițializarea rețelei.

### 2.3.2 Ecuția Bellman și Deep Q-Networks (DQN)

Pentru a afla politica optimă, agentul analizează fiecare acțiune posibilă folosind funcția valoare-acțiune (*Action-Value*)  $Q(s, a)$  care estimează recompensa așteptată pe termen lung. Actualizarea acestei funcții se face iterativ folosind Ecuția lui Bellman, care exprimă valoarea unei perechi stare-acțiune ca fiind suma dintre recompensa imediată și valoarea maximă estimată a stării următoare:

$$Q^*(s, a) = R(s, a, s') + \gamma \max_{a'} Q^*(s', a')$$

### Ecuția 2.3 Ecuția Bellman

În varianta clasică, algoritmul Q-Learning memorează valorile lui  $Q$  într-un tabel. Această abordare a problemei funcționează atunci când spațiul stărilor este redus. Cu toate acestea, când vine vorba



de monitorizarea traficului unei rețele, numărul de stări este foarte vast, deoarece pachetele de date pot avea numeroase combinații de caracteristici. Astfel, utilizarea unei reprezentări tabelare este imposibilă în contextul prezentat.

Pentru a rezolva această problemă, **Mnih et al.** [4] au propus arhitectura Deep Q-Network (DQN). DQN elimină tabelul și utilizează o rețea neuronală profundă (cu ponderile  $\theta$ ) pentru a aproxima direct valorile optime:  $Q(s, a; \theta) \approx Q^*(s, a)$ .

În această abordare, caracteristicile traficului de rețea reprezintă datele de intrare (*input*) ale rețelei, iar ieșirile (*output*) sunt valorile  $Q$  estimate pentru acțiunile disponibile (Permite sau Blochează). Această generalizare permite sistemului IDS să identifice corect tipare de atac chiar și în configurații de trafic pe care nu le-a întâlnit în timpul antrenamentului.

## 2.4 Stadiul actual al cercetării

Utilizarea algoritmilor de Învățare prin Recompensă în securitatea cibernetică a cunoscut o creștere semnificativă, devenind o alternativă tot mai promițătoare la metodele clasice, cum ar fi Arborii de Decizie, care au nevoie de seturi de date corecte și complet etichetate și care totodată tind să își piardă eficiența în medii dinamice, unde tiparele de atac evoluează constant.

Studii relevante, cum sunt cele realizate de **Caminero et al.** [5] și **Lopez et al.** [6], au demonstrat că agenții RL pot fi utilizați cu succes pentru a detecta traficul malițios, modelând interacțiunile dintre atacator și sistemul de detectare a intruzinilor ca un joc secvențial. Această perspectivă permite adaptarea continuă a mecanismelor de detecție în funcție de comportamentul adversarului.

Totuși analiza literaturii actuale evidențiază două limitări majore:

1. **Utilizarea datelor învechite:** O mare parte din studiile existente încă își evaluează agenții de tip RL folosind seturi de date învechite, precum KDD99 sau NSL-KDD. Aceste informații nu mai reprezintă infrastructura modernă a unei rețele și nu includ tipologii actuale ale unui atac cibernetic, cum ar fi DDoS de tip Slowloris sau vulnerabilitățile specifice aplicațiilor web.
2. **Funcții de recompensă suboptime:** Multe implementări se concentrează exclusiv pe maximizarea ratei de detecție (Recall), ignorând penalizarea adecvată a alarmelor false (False Positives), ceea ce face sistemele inaplicabile într-un mediu de producție real.

## 3. Dezvoltarea Sistemului de Detecție

Primul pas esențial în crearea sistemului inteligent de detecție a constat în organizarea setului de date. Pentru aceasta, a fost utilizat dataset-ul **CICIDS2017** [7], un standard contemporan pentru evaluarea sistemelor de securitate, care include atât trafic benign, cât și semnăturile celor mai frecvente atacuri cibernetice (Brute Force, DDoS, Atacuri Web, etc.).

Manipularea și curățarea volumului masiv de date s-au realizat utilizând biblioteca *Pandas* [8] din limbajul Python.

### 3.1 Pregătirea și preprocesarea datelor experimentale

Etapa inițială de preprocesare a datelor a constat în eliminarea instanțelor invalide (de tip *Inf* și *NaN*) și în uniformizarea setului de date, demers esențial pentru a asigura convergența și stabilitatea procesului de antrenare a agentului. Pentru compatibilizarea datelor cu arhitectura unui model de decizie binară, eticheta primară a fost recodificată: traficul de rețea legitim a fost asociat cu valoarea **0** („Permite”), în timp ce toate instanțele de trafic malițios, indiferent de tipologia atacului, au fost agregate sub valoarea **1** („Blochează”). În faza finală, având în vedere discrepanțele majore de scală dintre parametrii de rețea (variind de la zeci de octeți la milioane de microsecunde), s-a impus aducerea tuturor variabilelor continue în intervalul standardizat  $[0, 1]$ . Această operațiune a fost implementată utilizând clasa *MinMaxScaler* din cadrul bibliotecii *scikit-learn* [9]. Etapa de normalizare previne riscul ca algoritmul de învățare să fie predominat de valorile absolute mari și se realizează prin aplicarea următoarei transformări matematice asupra fiecărei valori  $x$  dintr-o caracteristică:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Ecuția 3.1 Normalizarea datelor

Unde  $x_{min}$  și  $x_{max}$  reprezintă minimul, respectiv maximul caracteristicii respective în setul de date. Efectul vizual al acestei transformări este ilustrat în Figura 3.1, unde se poate observa cum distribuția originală a datelor este perfect conservată, însă transpusă integral în noul interval standardizat. Prin parcurgerea acestui flux, vectorul de stări obținut devine strict numeric și echilibrat dimensional,

fiind complet pregătit pentru mediul de antrenament al agentului RL.

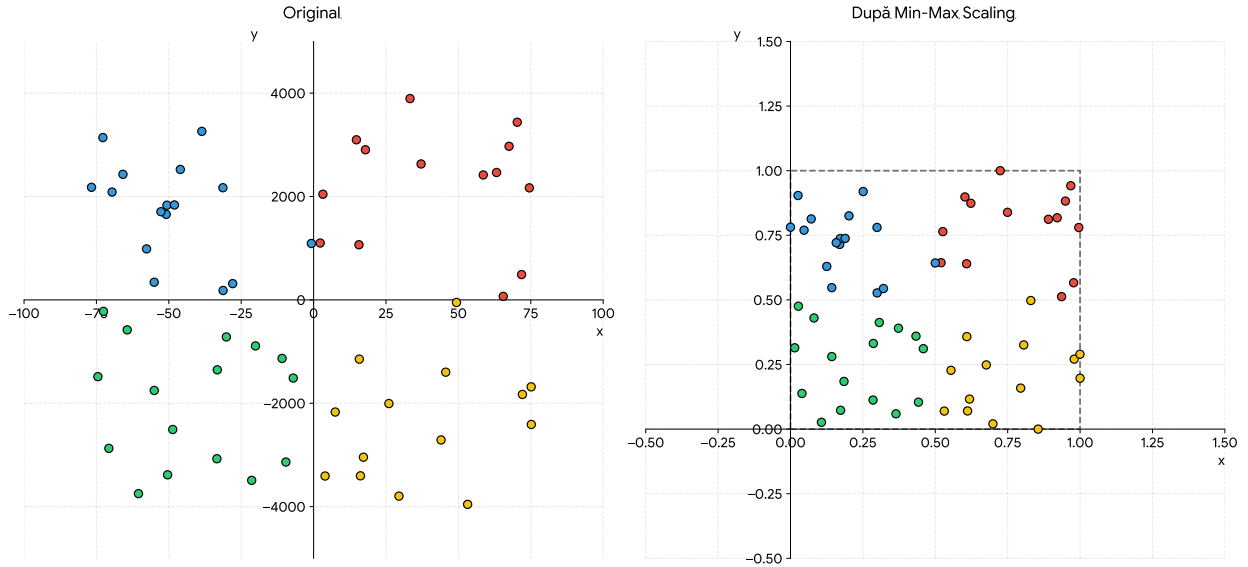


Figura 3.1: Exemplu de scalare a caracteristicilor

## 3.2 Proiectarea mediului de antrenament

Odată încheiată preprocesarea datelor, pasul următor a constat în crearea unei punți de comunicare între agentul decizional și datele de trafic. Astfel, a fost definit un mediu de învățare personalizat (*Custom Environment*), având ca punct de plecare structura standard oferită de biblioteca *Stable-Baselines3* [10] și respectând rigorile impuse de API-ul *Gymnasium* [11]. Prin adaptarea riguroasă a acestui cadru de lucru la specificul setului de date, problema detecției atacurilor de rețea a putut fi modelată conceptual sub forma unui Proces Decizional Markov (MDP).

### 3.2.1 Spațiul stărilor și spațiul acțiunilor

Pentru ca rețeaua neuronală să poată procesa informația, a fost necesară definirea limitelor mediului:

- **Spațiul stărilor:** A fost definit ca un spațiu continuu de tip *Box* (o zonă mărginită în spațiul  $n$ -dimensional). La fiecare pas  $t$ , starea  $s_t$  reprezintă un vector ce conține caracteristicile unui singur pachet de rețea. Datorită etapei anterioare de normalizare, s-a garantat că  $s_t \in [0, 1]^N$ , unde  $N$  este numărul de caracteristici (coloane) extrase din setul de date.
- **Spațiul acțiunilor:** Spațiul acțiunilor este discret, oferind agentului două decizii posibile la orice pas  $t$ : acțiunea  $a_t = 0$  (permite pachetul) și acțiunea  $a_t = 1$  (blochează pachetul).

### 3.2.2 Proiectarea funcției de recompensă

Nucleul mediului de antrenament, implementat în cadrul metodei  $step()$ , este reprezentat de funcția de recompensă  $R(s_t, a_t)$ . Aceasta a fost proiectată pentru a prioritiza detecția atacurilor, recunoscând că, în contextul securității cibernetice, omiterea unui atac real reprezintă un risc semnificativ mai mare decât generarea unei alarme false. Astfel, recompensa acordată agentului este guvernată de următoarea funcție matematică definită pe ramuri:

$$R(s_t, a_t) = \begin{cases} 1, & \text{dacă } a_t = y_t \text{ (Clasificare corectă)} \\ -2, & \text{dacă } a_t = 0 \text{ și } y_t = 1 \text{ (Atac ratat)} \\ -1, & \text{dacă } a_t = 1 \text{ și } y_t = 0 \text{ (Alarmă falsă)} \end{cases}$$

Ecuția 3.2 Recompensa agentului pe ramuri

Unde  $y_t$  reprezintă eticheta reală (cunoscută de mediu) a pachetului analizat la pasul  $t$ . Penalizarea asimetrică ( $-2$  pentru omiterea unui atac comparativ cu  $-1$  pentru generarea unei alarme false) forțează agentul să adopte o strategie ofensivă, acordând prioritate interceptării traficului malițios față de evitarea blocării traficului legitim. Această alegere de design reflectă realitatea operațională a sistemelor de detecție a intruziunilor, unde consecințele unui atac nedetectat sunt, în general, mai severe decât ale unui fals pozitiv.

### 3.2.3 Episoadele de antrenament și generalizarea

Pentru a preveni fenomenul de *overfitting* — situație în care agentul memorează secvența specifică a pachetelor din setul de date — antrenamentul a fost structurat în episoade a câte 1000 de pași. O provocare suplimentară a fost dezechilibrul accentuat al claselor din setul de date CICIDS2017, în care aproximativ 83% din instanțe reprezintă trafic normal. Fără o intervenție explicită, agentul ar fi expus preponderent la trafic benign, dezvoltând o preferință sistematică pentru acțiunea „permite”, ceea ce ar conduce la o rată ridicată de atacuri neclasificate.

Pentru a contracara acest fenomen, metoda *reset()* a fost adaptată să aplice o strategie de eșantionare echilibrată. La începutul fiecărui episod, cu o probabilitate de 50%, punctul de start este selectat aleatoriu din mulțimea instanțelor de atac, garantând că agentul este expus în mod echitabil atât la trafic normal, cât și la trafic malițios pe parcursul antrenamentului. Această inițializare stocastică controlată favorizează capacitatea de generalizare a agentului și reduce semnificativ rata atacurilor nedetectate.

---

**Algorithm 1** Logica de interacțiune și calculul recompensei

---

**Require:**  $a_t \in \{0, 1\}$  ▷ Acțiunea primită de la agent (0 = Permite, 1 = Blochează)

1:  $y_t \leftarrow$  eticheta reală a pachetului curent din setul de date

2:  $r_t \leftarrow 0.0$  ▷ Inițializarea recompensei

3: **if**  $a_t == y_t$  **then**

4:      $r_t \leftarrow 1.0$  ▷ Agentul a clasificat corect

5: **else if**  $a_t == 0$  **and**  $y_t == 1$  **then**

6:      $r_t \leftarrow -2.0$  ▷ Penalizare severă pentru atac ratat (Fals Negativ)

7: **else if**  $a_t == 1$  **and**  $y_t == 0$  **then**

8:      $r_t \leftarrow -1.0$  ▷ Penalizare pentru alarmă falsă (Fals Pozitiv)

9: **end if**

10:  $pas\_curent \leftarrow pas\_curent + 1$  ▷ Trecerea la următorul pachet de rețea

11: **if**  $pas\_curent \geq pasi\_maximi$  **then**

12:      $done \leftarrow \text{True}$  ▷ Episodul s-a încheiat

13:      $obs_{t+1} \leftarrow \vec{0}$  ▷ Stare terminală (vector nul)

14: **else**

15:      $done \leftarrow \text{False}$

16:      $obs_{t+1} \leftarrow$  caracteristicile pachetului de la  $pas\_curent$

17: **end if**

18: **return**  $obs_{t+1}, r_t, done$

---

### 3.3 Algoritmul Deep Q-Network (DQN)

În Capitolul 2 a fost introdusă ecuația lui Bellman, care definește valoarea optimă a unei perechi stare-acțiune ca suma dintre recompensa imediată și valoarea maximă estimată a stării următoare. Prezenta secțiune detaliază modul concret în care algoritmul DQN, propus de **Mnih et al.** [4], transformă acest fundament teoretic într-un proces de învățare funcțional, capabil să opereze în spațiul continuu și de dimensionalitate ridicată al caracteristicilor de trafic.

#### 3.3.1 De la ecuația Bellman la funcția de pierdere

Algoritmul clasic Q-Learning actualizează iterativ o tabelă de valori  $Q(s, a)$  pentru fiecare pereche stare-acțiune întâlnită. Însă când vine vorba de detecția intruziunilor, starea  $s_t$  este un vector continuu cu toate caracteristicile pachetului de rețea, ceea ce face imposibilă memorarea tuturor valorilor într-un tabel.

DQN rezolvă această problemă prin înlocuirea tabelii cu o rețea neuronală profundă parametrizată de ponderile  $\theta$ , care aproximează funcția de valoare optimă:  $Q(s, a; \theta) \approx Q^*(s, a)$ . Rețeaua primește la intrare starea  $s_t$  și produce la ieșire câte o valoare Q estimată pentru fiecare acțiune disponibilă.

Antrenarea rețelei se realizează prin minimizarea diferenței dintre valoarea  $Q$  prezisă de rețea și o valoare țintă derivată din ecuația Bellman. Această diferență, numită *Temporal-Difference Error*, este ridicată la pătrat pentru a forma funcția de pierdere:

$$\mathcal{L}(\theta) = E_{(s,a,r,s') \sim \mathcal{D}} \left[ \left( r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

Ecuția 3.3 Funcția de pierdere a algoritmului DQN

Unde  $\mathcal{D}$  reprezintă *replay buffer-ul*, iar  $\theta^-$  sunt ponderile rețelei țintă, două mecanisme esențiale ale DQN care vor fi detaliate în subsecțiunile următoare. Minimizarea funcției  $\mathcal{L}(\theta)$  se realizează prin *Stochastic Gradient Descent*, actualizând progresiv ponderile  $\theta$  ale rețelei în direcția care reduce eroarea de predicție.

### 3.3.2 Memoria de reluare (*Experience Replay*)

Un principiu fundamental al învățării supervizate este ca eșantioanele de antrenament să fie independente și identic distribuite (*i.i.d.*). Totuși, în învățarea prin recompensă, tranzițiile generate de interacțiunea agent-mediului sunt puternic corelate temporal: pachetele de rețea consecutive aparțin adesea aceleiași conexiuni sau aceluiași tip de atac, iar fiecare acțiune a agentului influențează starea pe care o va observa în pasul următor.

Pentru a elimina aceste corelații, DQN utilizează o **memorie de reluare** (*Experience Replay Buffer*), notată  $\mathcal{D}$ . La fiecare pas  $t$ , agentul stochează tranziția observată  $(s_t, a_t, r_t, s_{t+1})$  într-un buffer de dimensiune fixă. Ulterior, antrenamentul rețelei neuronale se realizează nu pe tranziția curentă, ci pe un **mini-batch aleatoriu** extras din  $\mathcal{D}$ .

Această abordare aduce două avantaje esențiale:

- **Eliminarea corelației temporale:** Prin eșantionarea aleatorie, tranzițiile dintr-un mini-batch provin din episoade și momente diferite, satisfacând aproximativ condiția *i.i.d.* și stabilizând procesul de învățare.
- **Eficiența utilizării datelor:** Fiecare tranziție poate fi reutilizată de mai multe ori în procesul de antrenare, ceea ce este deosebit de valoros atunci când anumite tipuri de trafic (de exemplu, atacurile rare) sunt slab reprezentate.

În implementarea curentă, biblioteca *Stable-Baselines3* alocă un buffer implicit de 1.000.000 de tranziții, asigurând o diversitate suficientă a experiențelor stocate pe parcursul celor 500.000 de

pași de antrenament.

### 3.3.3 Rețeaua țintă (*Target Network*)

Funcția de pierdere definită în Ecuația 3.3 conține atât valoarea prezisă  $Q(s, a; \theta)$ , cât și valoarea țintă  $r + \gamma \max_{a'} Q(s', a'; \theta)$ . Dacă ambele ar fi calculate cu aceeași rețea, fiecare actualizare a ponderilor  $\theta$  ar modifica simultan atât predicția, cât și ținta, generând un fenomen de instabilitate cunoscut drept „urmărirea unei ținte în mișcare” (*moving target problem*).

Pentru a contracara această instabilitate, DQN introduce o a doua copie a rețelei neuronale, numită **rețeaua țintă**, cu ponderile  $\theta^-$ . Această rețea este identică structural cu rețeaua principală, dar ponderile sale sunt **înghețate** și actualizate periodic, prin copierea directă a valorilor din rețeaua principală la intervale regulate de  $\tau$  pași:

$$\theta^- \leftarrow \theta \quad \text{la fiecare } \tau \text{ pași}$$

Ecuația 3.4 Actualizarea periodică a rețelei țintă

Între aceste sincronizări, rețeaua țintă oferă valori stabile pentru calculul obiectivului de antrenare, ceea ce permite convergența algoritmului. În configurația curentă, biblioteca *Stable-Baselines3* realizează implicit această sincronizare la fiecare 10.000 de pași de antrenament.

Algoritmul 2 sintetizează întregul flux de antrenare al agentului DQN, integrând cele trei mecanisme prezentate: memoria de reluare, rețeaua țintă și minimizarea funcției de pierdere.

### 3.3.4 Politica de explorare $\varepsilon$ -greedy

Un agent de Reinforcement Learning se confruntă permanent cu **dilema explorare-exploatare**: dacă alege întotdeauna acțiunea cu valoarea  $Q$  maximă (exploatare pură), riscă să rămână blocat într-o politică suboptimală, deoarece nu va descoperi niciodată acțiuni potențial mai bune pe care nu le-a încercat. Strategia  $\varepsilon$ -greedy rezolvă această dilemă într-un mod simplu, dar eficient:

$$a_t = \begin{cases} \text{acțiune aleatorie din } A, & \text{cu probabilitatea } \varepsilon \\ \arg \max_a Q(s_t, a; \theta), & \text{cu probabilitatea } 1 - \varepsilon \end{cases}$$

Ecuația 3.5 Politica de selecție  $\varepsilon$ -greedy

---

**Algorithm 2** Algoritmul de antrenare DQN

---

```
1: Inițializează rețeaua principală  $Q(s, a; \theta)$  cu ponderi aleatorii
2: Inițializează rețeaua țintă  $Q(s, a; \theta^-)$  cu  $\theta^- \leftarrow \theta$ 
3: Inițializează memoria de reluare  $\mathcal{D} \leftarrow \emptyset$ 
4: for fiecare pas de antrenament  $t = 1, 2, \dots, T$  do
5:   Observă starea curentă  $s_t$ 
6:   Selectează acțiunea  $a_t$  utilizând politica  $\varepsilon$ -greedy (Ecuația 3.5)
7:   Execută  $a_t$  în mediu, observă recompensa  $r_t$  și noua stare  $s_{t+1}$ 
8:   Stocchează tranziția  $(s_t, a_t, r_t, s_{t+1})$  în  $\mathcal{D}$ 
9:   Extrage un mini-batch aleatoriu de tranziții din  $\mathcal{D}$ 
10:  Calculează ținta:  $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ 
11:  Actualizează  $\theta$  prin minimizarea pierderii  $(y - Q(s, a; \theta))^2$ 
12:  if  $t \bmod \tau == 0$  then
13:    Sincronizează rețeaua țintă:  $\theta^- \leftarrow \theta$ 
14:  end if
15: end for
```

---

Cu o probabilitate  $\varepsilon$ , agentul alege o acțiune complet aleatorie (explorare), iar cu probabilitatea  $1 - \varepsilon$ , selectează acțiunea care maximizează funcția Q curentă (exploatare). Valoarea  $\varepsilon$  nu rămâne constantă pe parcursul antrenamentului, ci urmează un program de **descreștere liniară**:

$$\varepsilon(t) = \max \left( \varepsilon_{final}, \varepsilon_{init} - \frac{\varepsilon_{init} - \varepsilon_{final}}{T_{explorare}} \cdot t \right)$$

Ecuația 3.6 Descreșterea liniară a ratei de explorare

În configurația sistemului propus,  $\varepsilon_{init} = 1.0$  (explorare totală la începutul antrenamentului),  $\varepsilon_{final} = 0.05$  și  $T_{explorare} = 150.000$  pași (30% din totalul de 500.000 pași). Concret, în primii 150.000 de pași, agentul reduce progresiv componenta aleatorie, iar după acest prag, menține un nivel minim de explorare de 5%, ceea ce îi permite să continue descoperirea unor configurații rare de trafic pe care nu le-a întâlnit frecvent.

## 3.4 Arhitectura și Configurarea Agentului

### 3.4.1 Arhitectura rețelei neuronale

Rețeaua neuronală care aproximează funcția  $Q(s, a; \theta)$  este un **Perceptron Multistrat** (*Multi-Layer Perceptron — MLP*), o arhitectură de tip *feedforward* în care informația se propagă de la stratul de intrare către stratul de ieșire, traversând mai multe straturi ascunse intermediare. Arhitectura



completă este prezentată în Figura ?? și se compune din următoarele componente:

- **Stratul de intrare:** Primește vectorul de stare  $s_t \in [0, 1]^N$ , unde  $N$  reprezintă numărul de caracteristici extrase din setul de date CICIDS2017 (corespunzând atributelor numerice ale fiecărui pachet de rețea).
- **Straturi ascunse:** Rețeaua conține trei straturi ascunse cu dimensiunile  $[256, 256, 128]$  neuroni. Fiecare strat aplică o transformare liniară urmată de funcția de activare *ReLU* (*Rectified Linear Unit*):

$$h_l = \text{ReLU}(W_l \cdot h_{l-1} + b_l) = \max(0, W_l \cdot h_{l-1} + b_l)$$

Ecuția 3.7 Transformarea aplicată de fiecare strat ascuns

Unde  $W_l$  și  $b_l$  sunt matricea de ponderi, respectiv vectorul de *bias* al stratului  $l$ , iar  $h_{l-1}$  este ieșirea stratului anterior (sau vectorul de intrare  $s_t$  pentru primul strat). Funcția ReLU introduce **neliniaritate** în model, permițându-i să captureze relații complexe între caracteristicile pachetelor care nu ar putea fi reprezentate printr-o simplă combinație liniară.

- **Stratul de ieșire:** Produce doi neuroni, coresponzând valorilor  $Q$  estimate pentru cele două acțiuni disponibile:  $Q(s_t, 0; \theta)$  (permite pachetul) și  $Q(s_t, 1; \theta)$  (blochează pachetul). Acțiunea selectată de agent este:

$$a_t = \arg \max_{a \in \{0,1\}} Q(s_t, a; \theta)$$

Ecuția 3.8 Selecția acțiunii pe baza valorilor  $Q$

Această configurație cu trei straturi ascunse a fost aleasă deoarece topologia implicită a bibliotecii *Stable-Baselines3* (două straturi de 64 de neuroni) nu oferă suficientă capacitate de reprezentare pentru spațiul de observație de dimensionalitate ridicată al setului CICIDS2017.

### 3.4.2 Sinteza hiperparametrilor

Toți parametrii principali utilizați în configurarea agentului sunt sintetizați în Tabelul 3.1. Fiecare parametru a fost detaliat în secțiunile anterioare.

| Parametru                         | Valoare         | Justificare                              |
|-----------------------------------|-----------------|------------------------------------------|
| Algoritm                          | DQN             | Eficient pentru acțiuni discrete         |
| Politică                          | MlpPolicy       | Spațiu de observație continuu            |
| Arhitectură rețea                 | [256, 256, 128] | Dimensionalitate ridicată a intrărilor   |
| Rată de învățare ( $\alpha$ )     | 0.001           | Standard pentru DQN                      |
| Factor de discount ( $\gamma$ )   | 0.95            | Convergență stabilă pe termen mediu      |
| Fracțiune explorare               | 0.30            | Explorare extinsă în faza inițială       |
| $\epsilon_{final}$                | 0.05            | Menținerea unui nivel minim de explorare |
| Dimensiune replay buffer          | 1.000.000       | Diversitate ridicată a experiențelor     |
| Frecvență sincronizare $\theta^-$ | 10.000 pași     | Stabilitatea țăintelor de antrenare      |
| Pași de antrenament               | 500.000         | Convergență completă a politicii         |
| Pași per episod                   | 1.000           | Varietate maximă a instanțelor văzute    |

Tabela 3.1: Hiperparametrii agentului DQN

### 3.5 Rezultatele Antrenamentului și Evaluarea Performanței

După finalizarea procesului de antrenament pe parcursul a 500.000 de pași, agentul DQN a fost evaluat pe întregul set de date CICIDS2017. Figura ?? ilustrează evoluția acurateței de-a lungul antrenamentului, iar Figura ?? prezintă matricea de confuzie obținută în urma evaluării finale.

Rezultatele confirmă că agentul a învățat să clasifice eficient traficul de rețea.

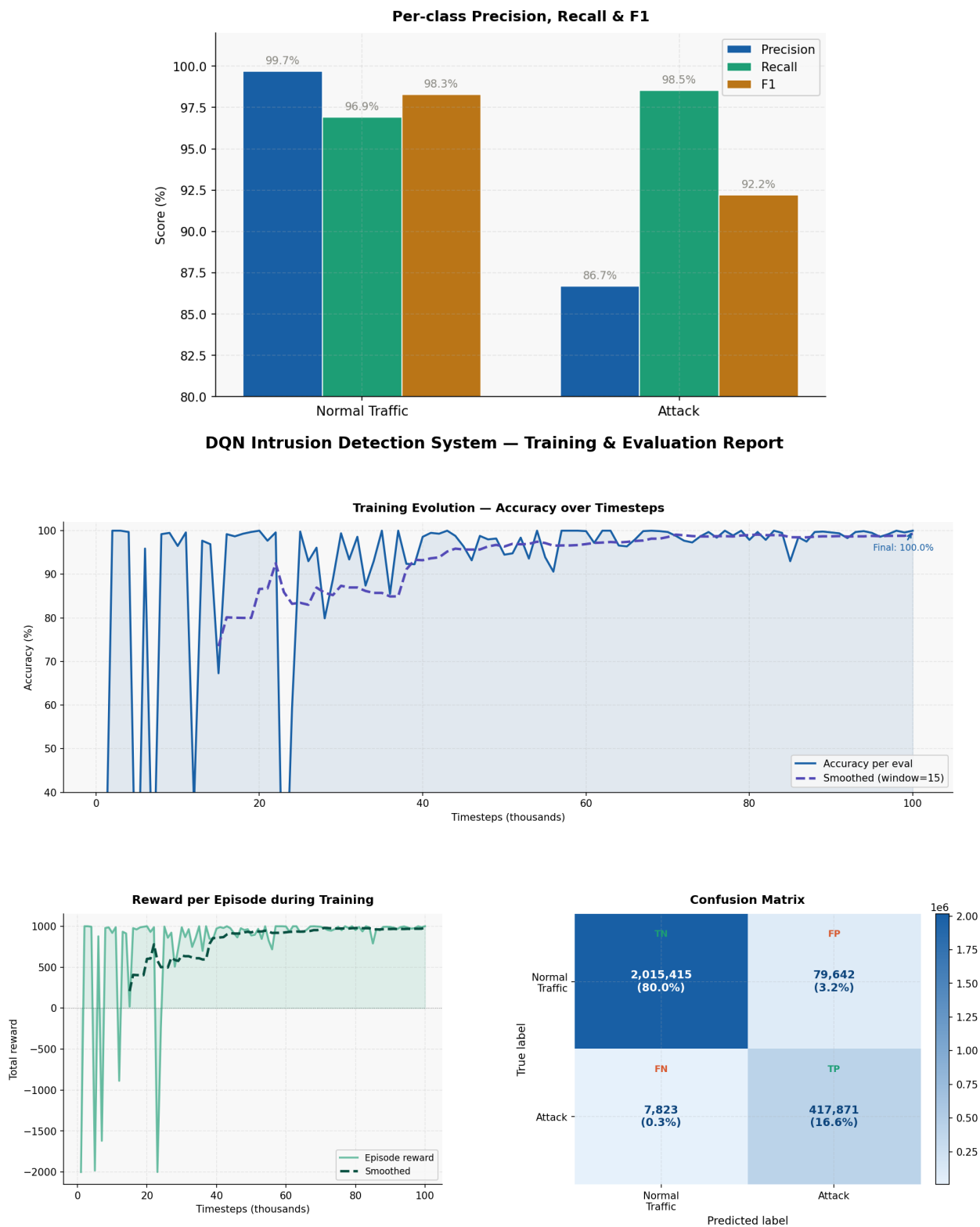


Figura 3.2: Rezultatele evaluării agentului DQN

## 4. Concluzii

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean dignissim metus justo, nec pharetra mauris tincidunt id.

## 5. Bibliografie

- [1] A. V. Aho and M. J. Corasick, “Efficient string matching: an aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [2] D. E. Denning, “An intrusion-detection model,” *IEEE Transactions on software engineering*, no. 2, pp. 222–232, 1987.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, second ed., 2018.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] G. Caminero, M. Lopez-Martin, and B. Carro, “Adversarial environment reinforcement learning algorithm for intrusion detection,” *Computer Networks*, vol. 159, pp. 96–109, 2019.
- [6] M. Lopez-Martin, B. Carro, and A. Sanchez-Esguevillas, “Network intrusion detection based on reinforcement learning and deep generative models,” *IEEE Access*, vol. 8, pp. 151474–151486, 2020.
- [7] Canadian Institute for Cybersecurity (CIC), “Intrusion detection evaluation dataset (cic-ids2017).” <https://www.unb.ca/cic/datasets/ids-2017.html>, 2017. Universitatea din New Brunswick (UNB). [Accesat la: 18.02.2026].
- [8] The pandas development team, “pandas-dev/pandas: Pandas.” <https://doi.org/10.5281/zenodo.18675244>, Februarie 2020. Versiunea 3.0.1. DOI: 10.5281/zenodo.18675244 [Accesat la: 25.02.2026].
- [9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

- [11] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.